

ZLC FPGA Pulse Streamer 手册

Artix-7 35T 边沿流送器 / 1-tick 预取 / 无限流式扫描 / JTAG-to-AXI

RTL、1-tick 预取、双 bank 流式、仿射扫描引擎、模拟总线 DAC、编译上传流
程、资源预算

2026-06-07

Contents

1	写给新读者	1
1.1	这块硬件是什么	1
1.2	两句话能力总结	1
1.3	文件	2
2	架构总览	3
2.1	从主机到引脚的一条链	3
2.2	CTRL 寄存器邮箱	3
2.3	一行边沿的含义	4
2.4	时钟与对齐	4
3	1-tick 预取流水线	5
3.1	为什么需要预取	5
3.2	解法: 深度 FIFO_DEPTH 的连续预取 + 影子	5
3.3	背靠背 1-tick 边沿: 前端脉冲实绘	5
4	无缝边界: 四个 gapless 重装点	7
4.1	边界为什么不留缝	7
4.2	扫描点 $k \rightarrow k+1$ 跨界无缝: 前端脉冲实绘	7
5	无限流式扫描: 2-bank ping-pong	9
5.1	窗口 + 流式回填	9
5.2	bank_chunk 握手与晚到 STALL	9
5.3	repeat_forever 流式重扫	10
6	N-slot 仿射扫描引擎	11
6.1	核心公式	11
6.2	为什么这是对的取舍	11
7	模拟总线 DAC 引擎	12
7.1	段表编码	12
7.2	模拟 DAC ramp 与扫描端点: 前端脉冲实绘	12

8	容量与资源预算	14
8.1	35T 上解出的容量.	14
9	编译与上传流程	15
9.1	build / program / run.	15
9.2	一次上传里发生了什么	15

1

写给新读者

1.1 这块硬件是什么

pulse-streamer 是一个**固定 bitstream + 运行时表**的脉冲序列器。烧一次 bitstream 就得到时钟状态机、板上输出引脚、`jtag_axi + axi_bram_ctrl` 数据通路和片上 BRAM；它**不**把某个具体实验脉冲写死进 Verilog。之后每次 `On Pulse`，主机都把当前的 `PulseTableState/PulseSequence` 编译成一张紧凑的程序映像，经 **JTAG-to-AXI** 写进 FPGA 的 BRAM，再用一个 CTRL 寄存器邮箱 (COMMAND/STATUS) 控制引擎运行。

这是**唯一**一套设计：没有变体、没有向后兼容、没有探针控制面、没有逐通道游程引擎、没有加载器 FSM。整条路径就是“主机打包 BRAM 映像 → AXI 写入 → CTRL 邮箱命令 → 边沿表引擎播放”。

目标器件

本设计面向 **Xilinx Artix-7 35T (xc7a35tfgg484-2)**，FGG484 封装。这是一颗 LUT/BRAM 资源有限的小器件，所以所有容量参数都按它来权衡。

近似资源类别：

xc7a35tfgg484-2

logic cells: 33,280	CLB LUTs: 20,800	CLB FFs: 41,600
BRAM: 1,800 Kb (50 x 36 Kb RAMB36)	DSP: 90	

1.2 两句话能力总结

- **最小脉冲宽度与分辨率 = 1 个 tick (20 ns)**。背靠背的 1-tick 边沿能逐周期、一拍一个地打出来——靠的是一条连续预取流水线把 BRAM 读延迟藏掉（见第三章）。
- **扫描点数无上限**。常驻一个 2-bank ping-pong 窗口 (4096 点)，主机在游标后面把空出来的 bank 流式回填下一段（见第五章）。配合 N-slot 仿射扫描，duration / delay / DAC 值可以在**一次连续运行里任意组合地无缝扫描**。

1.3 文件

fpga/pulse_streamer/zlc_edge_streamer.v 引擎。全局边沿表 (三块并行 BRAM: tick 32b / coeff 64b / mask 62b, 强制 `READ_LATENCY_B=2`)、深度 `FIFO_DEPTH(=RD_LAT+1=3)` 的连续边沿预取、2-bank ping-pong 扫描窗口、仿射有效 tick MAC、模拟总线 DAC 引擎。

fpga/pulse_streamer/zlc_pulse_streamer_top.v 顶层。`jtag_axi` → `axi_bram_ctrl` → 区段译码的 BRAM (三块并行边沿 + 扫描窗口 + 总线映像) + CTRL 寄存器邮箱, 驱动引擎与板上 62 路输出 + 4x10b DAC 总线。

fpga/pulse_streamer/host/image.py 主机 ↔ FPGA AXI 写契约 + 容量求解器的单一真相源。`pack_program/unpack_program` 打包/解码 BRAM 映像, `solve_capacity` 从器件 RAMB36/LUT 预算推导几何。RTL localparam 与 create-project Tcl 都从它派生。

fpga/pulse_streamer/host/engine_model.py 逐周期行为模型 (仓库没有 Verilog 仿真器, 这是硬件前的正确性证明)。

Zou_lab_control/neutral_atom/devices/axi_session.py 主机会话 `VivadoAxiStreamerSession`: 常驻 Vivado `hw_axi`, 打包上传 + 驱动 CTRL 邮箱 + 流式回填。

2

架构总览

2.1 从主机到引脚的一条链

真实路径是双电脑结构。控制电脑通过 RPyC 把命令发到 FPGA 电脑上的 server; server 持有一个常驻 Vivado `hw_axi` 会话, 用 JTAG-to-AXI 把程序映像写进 FPGA 的 BRAM, 再写 CTRL 寄存器命令引擎运行。

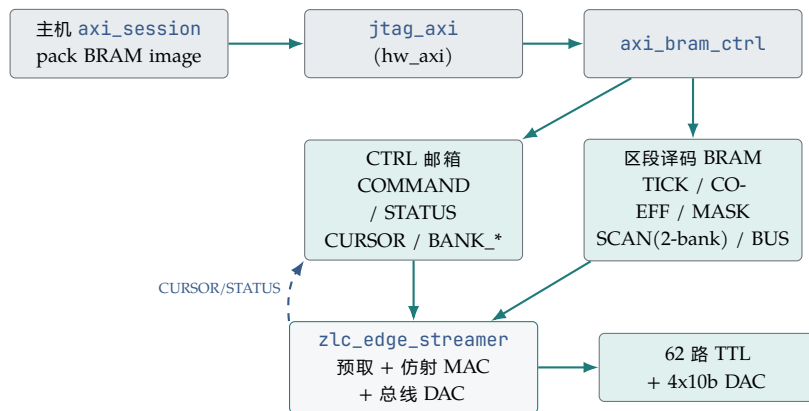


Figure 2.1: 数据通路: 主机经 JTAG-to-AXI 写区段译码的 BRAM 与 CTRL 邮箱; 引擎从 BRAM 播放, 并把 STATUS/CURSOR 回写邮箱供主机轮询/流式回填。

2.2 CTRL 寄存器邮箱

主机不碰任何探针; 它只读写一小块 CTRL 寄存器文件 (来自 `host.image.CtrlWords`):

CTRL mailbox			
COMMAND	主机->top	上升沿	LOAD(1) / FIRE(2) / RESET(4) / SAFE(8)
STATUS	top->主机		LOADED(1)/RUNNING(2)/DONE(4)/ERROR(8)/UNDERFLOW(16)
PROG_COUNT		边沿行数	
SCAN_COUNT		扫描点总数 N	(可超过常驻窗口)

```

SCAN_ENABLE / REPEAT_FOREVER
LOOP_START / LOOP_COUNT / LOOP_END_TICK / LOOP_END_LO / LOOP_END_HI
BUS_COUNTS / BANK_SIZE / SLOT_COUNT
CURSOR      top-> 主机    已消费的扫描点数（驱动流式回填）
BANK_READY  主机->top    bit b = bank b 已装好
BANK0_CHUNK / BANK1_CHUNK 主机->top  各 bank 当前装的是哪一段 chunk

```

生命周期: `prepare` (SAFE、上传映像、arm bank、LOAD) / `fire` (FIRE) / `wait_done` (轮询 STATUS、跟随 CURSOR 流式回填) / `safe_state` (SAFE)。COMMAND 字在每个命令前先清 0，制造干净的上升沿。

2.3 一行边沿的含义

引擎里保存一张全局边沿表，每行三个字段，分别存在三块**并行的** BRAM 里：

每行边沿

```

tick[i]   : 该边沿的基准 FPGA tick (32 bit)           -- TICK BRAM
coeff[i]  : NUM_SLOTS 个定点扫描系数 (NUM_SLOTS*16 bit) -- COEFF BRAM
mask[i]   : 该 tick 上完整的输出状态 (62 bit)         -- MASK BRAM

```

bit 0 对应 `channels[0]`。一行的含义是：**在这个有效 tick，把所有输出切换成这个 mask**——不是逐 channel 命令。三块 BRAM 并行读，所以一次访问拿到一整行边沿，无需位宽填充；`max_edges` 因此可以做大（35T 上 4096 行）。

2.4 时钟与对齐

真实硬件默认 50 MHz，一个 tick 是 20 ns。主机在上传前校验所有 period 持续时间、channel 延时、扫描点都对齐到这个 tick 网格 (`1e9 / clock_hz`)。32 位 tick 计数器在 50 MHz 下覆盖约 85.9 秒。同步后的 FIRE 路径会引入一个固定的小启动延迟，但那是常数偏移，不是比例因子；脉冲宽度和延时由 tick 计数决定。

3

1-tick 预取流水线

3.1 为什么需要预取

边沿表放在 block RAM 里, 而 block RAM 是**同步读**: 发出读地址后, 数据要 RD_LAT 个周期后才到。build tcl 用 `zlc_force_latency2` 把边沿 BRAM 强制成 $READ_LATENCY_B=2$, 所以 $RD_LAT=2$ 是确定的。如果天真地“到点了才去读下一行”, 每条边沿之间就至少要空 2 个周期——背靠背的 1-tick 边沿根本打不出来。

3.2 解法: 深度 FIFO_DEPTH 的连续预取 + 影子

引擎维护一个深度 $FIFO_DEPTH(=RD_LAT+1=3)$ 的“arm” FIFO, 里面装着**接下来要打的几条边沿**。它**每个周期发一次 BRAM 读**, 把读回的边沿压进 FIFO 尾; 当 `time_count` 追上 FIFO 头那条边沿的有效 tick 时, 就在**同一周期**把它的 mask 打到输出、并把它弹出。因为 FIFO 深度比读延迟多 1, FIFO 永远不空, 背靠背的 1-tick 边沿就能一拍一个地连续命中。

SEED 不变式 (行为模型逼出来的关键)

在每个边界, 从“第一条还没输出的边沿”开始**播种 FIFO_DEPTH 条影子**, 并且边界当拍**不发新读** (占用 == 影子数 $\leq$ 深度)。3 条影子刚好够给紧跟着的 1-tick 后继留出时间; 2 条影子会少一拍而丢边沿。这条不变式是逐 tick 镜像仿真“逼”出来的。

3.3 背靠背 1-tick 边沿: 前端脉冲实绘

边沿表放在 block RAM(同步读, $RD_LAT=2$), 引擎用一条深度 $FIFO_DEPTH=RD_LAT+1=3$ 的连续预取流水线把读延迟藏掉: 它**每周期发一次 BRAM 读** (周期 T 发出的读, 数据在 $T+RD_LAT$ 落地), 读回的边沿压进 arm FIFO 尾; 当 `time_count` 追上 FIFO 头那条边沿的有效 tick 时, 在**同一周期**把它的 mask 打到输出并弹出。因为 FIFO 深度比读延迟多 1, 弹出与补充始终咬合、FIFO 不空。下图是**前端脉冲绘图器实绘**的引擎输出: 三路通道在相邻 20 ns tick 上背靠背切换, 每个 1-tick 边沿逐拍打出、中间无空拍——这正是 20 ns 的最小脉宽与分辨率。

- 周期 $-2, -1$ 是**arm 阶段** (reset/LOAD 期间): 连发读把前几条边沿的影子预取进 FIFO, 使 `time_count` 从 0 开始时 FIFO 已经有货。

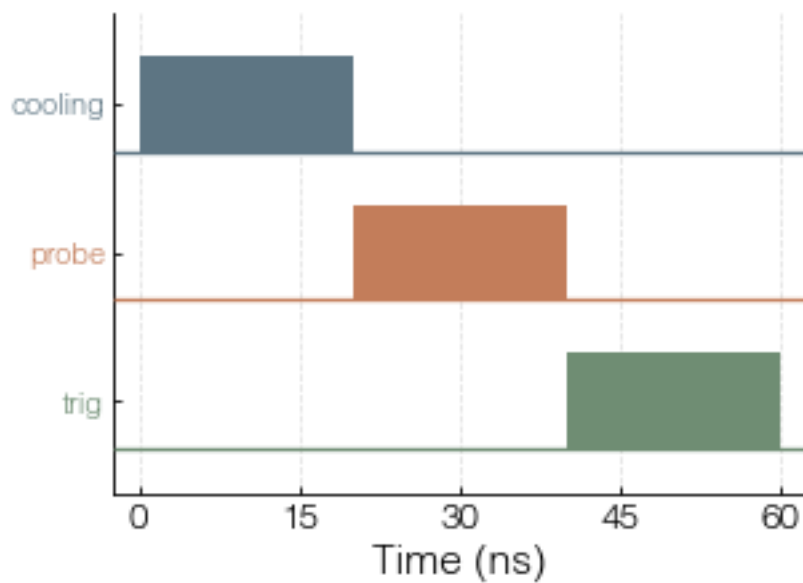


Figure 3.1: 前端脉冲实绘: 三路通道在相邻 20 ns tick 上背靠背切换——引擎的最小脉宽与分辨率就是 1 个 tick; 预取流水线让这些 1-tick 边沿逐拍打出, 中间无空拍。

- tick 0: FIFO 头是 A 且 `time_count=tick[A]=0`, 当拍输出 A 并弹出; 同一拍 C 的数据正好落地补进 FIFO。
- 由于 FIFO 深度 3 > 读延迟 2, 弹出与补充永远咬合, tick 1/2/3 连续命中 B/C/D, 无空拍。

这条等价是被证明的

`host.engine_model.rtl_mirror_play` 在读延迟 1/2/3 下逐 tick 等于组合参考播放器 `reference_play`, 并跑了 200 个 fuzz 程序 (含背靠背 1-tick、扫描 1-tick、loop 1-tick)。仓库无 Verilog 仿真器, 这个逐周期镜像就是硬件前最强证据。

4

无缝边界：四个 gapless 重装点

4.1 边界为什么不留缝

引擎只有一个全局边沿指针，所以一次 repeat / loop-rewind / scan-advance / start 只是单周期的指针 + 影子重装。四个 gapless 重装点 (start / loop-rewind / scan-advance / repeat) 在边界用上一节的 SEED 不变式重新播种 FIFO_DEPTH 条影子，于是点 k 的最后一条边沿与点 $k+1$ 的第一条边沿在时间上紧挨着，中间没有缝。

4.2 扫描点 $k \rightarrow k+1$ 跨界无缝：前端脉冲实绘

仿射扫描把 duration/delay/DAC 绑到 slot $s_0 \dots s_N$ ；每个扫描点用新的 slot 向量，边沿的有效 tick = $\text{base} + (\text{sum coeff} * \text{slot}) \gg \text{FRAC}$ 随之平移。下图是前端实绘的同一脉冲在两个扫描点的输出——被扫的中间周期把后面的边沿在硬件里 lockstep 平移。引擎在播放点 k 末尾的同一拍，就已按 $k+1$ 的新 slot 向量把首批影子重新播种好，所以点 k 的末边沿与点 $k+1$ 的首边沿在时间上紧挨着、无 gap。

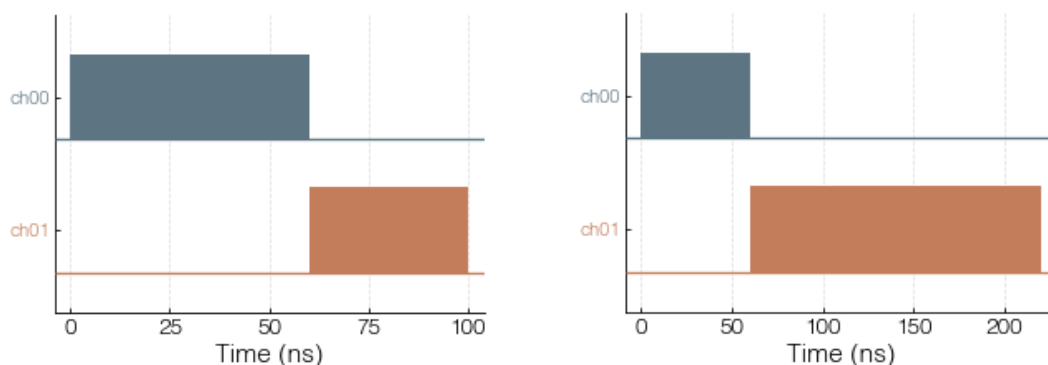


Figure 4.1: 前端脉冲实绘：同一脉冲在两个 scan 点的渲染。被扫的中间周期把后面的边沿在硬件里 lockstep 平移；扫描点之间的切换是无缝的（边界影子重装）。

- 边界处引擎不发新读，只用已经播种好的影子。新 slot 向量在边界当拍生效，所以下一点首条边沿的有效 tick 已按 $k+1$ 的扫描值算好。

- loop-rewind(repeat bracket) 走同一套机制: 指针跳回 `loop_start_addr`, 影子按 loop 起点重新播种, 循环接缝同样无缝。下图是前端实绘的硬件重复 loop 循环体 (不展开), 重复之间无缝回绕。

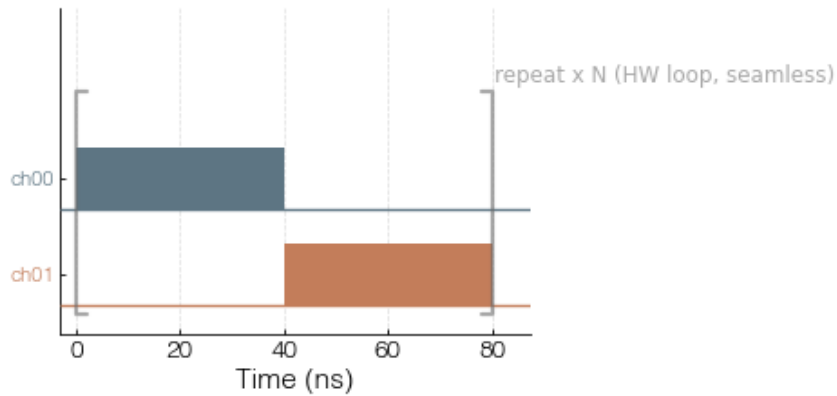


Figure 4.2: 前端脉冲实绘: 硬件重复 loop 的循环体 (不展开)。repeat_forever 在硬件里无缝回绕, 重复之间不留缝。

host.engine_model 用 streaming_scan_play 与 1-tick 边界 fuzz 证明了这一点: 跨扫描点 / 跨 loop 的边沿与不分段的参考播放逐 tick 相同。

5

无限流式扫描：2-bank ping-pong

5.1 窗口 + 流式回填

扫描窗口在硬件里是一个 **2-bank ping-pong** ($BANK_SIZE=2048$ ，是 2 的幂；两 bank 共 4096 个常驻扫描点)，放在一块 BRAM 里。引擎依次播放点 $0..N-1$ ，用 $(idx/BANK_SIZE) \bmod 2$ 选 bank，并把已消费点数写进 **CURSOR**。主机读 **CURSOR**，把游标已经走过的那个 bank 回填**下一段 chunk**，并更新 **BANK_READY/BANK*_CHUNK**。这样扫描点文件虽然有限，但**大小无上限**。



Figure 5.1: 2-bank ping-pong: 引擎播放当前 bank、推进 CURSOR; 主机在游标背后把空出来的 bank 回填下一段 chunk。

5.2 bank_chunk 握手与晚到 STALL

引擎只在 **BANK_READY** 为真且该 bank 装的是**正确 chunk** ($BANK_CHUNK$ 与引擎期望的段号一致) 时才推进。如果回填来晚了，引擎**停住** (hold 在当前点，置 **STATUS_UNDERFLOW**)，**绝不**播放错误或过期的点；主机回填到位后它再继续。这把“晚到”变成一次安全的 hold，而不是数据损坏。

streaming refill timeline + late-refill STALL

```
CURSOR (pts done):  0 ..... 2047 | 2048 ..... 4095 | 4096 .....
engine plays bank:  bank0          | bank1          | bank0(chunk2)
BANK_READY         :  b0,b1 ready  | b0 freed       | refill in flight
BANK0_CHUNK        :  0            | (host -> 2)    | 2
host refill bank0  :  [.... writing chunk 2 ....]
                    ^
case A 及时:  chunk2 在游标到 4096 前写好 -> BANK0_CHUNK=2 ready -> 无缝继续
case B 晚到:  游标到 4096 但 bank0 还不是 chunk2
               -> 引擎 STALL (hold 当前点, STATUS_UNDERFLOW=1)
```

```
-> 主机写完 chunk2 + BANK0_CHUNK=2 + BANK_READY  
-> 引擎自动继续，不丢点/不串点
```

5.3 repeat_forever 流式重扫

`repeat_forever` 对一个有限（但很大）的流式扫描做反复扫掠：`fire` 时启动一个主机**后台回填线程**，循环把下一段填进空出的 bank。一次扫掠之内是**无缝**的；扫掠与扫掠之间的接缝是一个**短暂的安全 hold**（重新从 chunk 0 播种）。`host.engine_model.streaming_scan_play` 证明了播放序列、CURSOR 推进、以及晚到 STALL 行为。

6

N-slot 仿射扫描引擎

6.1 核心公式

每行边沿存一个 base tick 加 `NUM_SLOTS` 个定点系数；运行时 FPGA 对每个扫描点计算：

effective tick

```
effective_tick = base_tick + (sum_j coeff_j * slot_j) >>> COEFF_FRAC_BITS
```

其中 `slot_j` 是当前扫描点第 j 个 slot 的值（来自扫描点表那一行），`COEFF_FRAC_BITS=8` 是定点小数位数。**已经没有 x/y** ——slot 是任意多个命名维度 `s0, s1, ...`，每个绑定一个 duration / delay / DAC 值字段。这同一个 `effective_tick` 表达式被边沿 tick、loop 终点 tick、以及总线段起止 tick **复用**——这正是为什么 duration、delay、DAC 值能任意组合地同时扫描，且只花一条 MAC 的 LUT。

6.2 为什么这是对的取舍

spec 最初设想的是逐通道游程编码（per-channel）+ 双 bank 全新存储。我们改成把仿射引擎**泛化**成 N 个 slot 的流式扫描点表，原因围绕 35T 的现实：

- 存储复杂度 $O(\text{edges}) + O(\text{points} \times \text{slots})$ 已达成 spec 的核心存储目标，但不需要每通道游程的状态机与地址逻辑。
- LUT 最省：一条比较器 + 一条仿射 MAC，而不是 62 个通道 player。
- 可验证：在无 Verilog 仿真器的前提下，仿射方案能被纯 Python 的逐周期行为模型忠实复刻。

7

模拟总线 DAC 引擎

7.1 段表编码

每条 10-bit DAC 总线有一张小的 LUTRAM 段表 (每 tick 组合读), 每段编码:

bus segment

```
start_tick / stop_tick : 段的起止 tick (仿射, 带 NUM_SLOTS 个系数, 同  
↪ effective_tick)  
start_value / stop_value: 段起止 DAC 码 (10 bit)  
mode                  : edge/hold 或 ramp  
value_select (dual)   : start 与 stop 各一个选择子
```

edge/hold 段把输出保持在某个值; **ramp** 段在 start 到 stop 之间**线性插值**, 引擎本地每步走 1 个码 (不额外占边沿行)。 **dual value_select** 让 start 与 stop 各自可以指向一个扫描 slot——于是一条 ramp 能**同时扫两个端点** (扫描-A → 扫描-B), 一条 edge/hold 段能跟随一个扫描的 DAC 码。

7.2 模拟 DAC ramp 与扫描端点: 前端脉冲实绘

每条模拟总线是一张小 LUTRAM 段表 (组合读), 段含 start/stop tick(仿射系数)+ start/stop 值 + mode(edge/ramp)+ focus 双 value_select(start 和 stop 各一个)。ramp 段由引擎在本地按 tick 插值生成 10-bit 阶梯 (不占数字边沿行); 双 value_select 让一条 ramp 的 focus 两个端点各跟一个 scan slot(scanned-A o scanned-B), edge/hold 段则跟单个被扫 DAC 码。下图是 focus 前端脉冲绘图器实绘的 DAC 波形。

`host.engine_model.bus_play` 逐 tick 证明了总线/ramp 引擎, 包含被扫描的 ramp 端点。

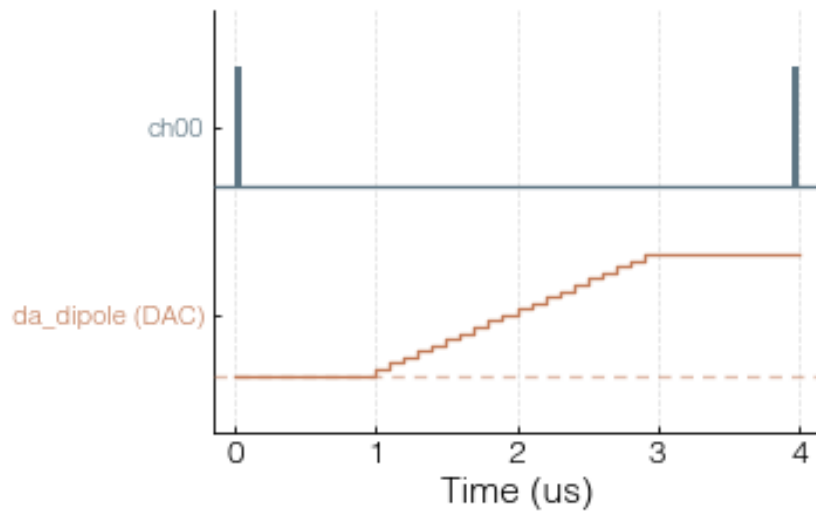


Figure 7.1: 前端脉冲实绘: 模拟总线 DAC 波形——保持 0、斜坡 0→1023、保持 1023。引擎在本地按 tick 插值生成 10-bit 阶梯; 双 value_select 允许斜坡两端各跟一个 scan slot。

8

容量与资源预算

8.1 35T 上解出的容量

容量由 `host.image.solve_capacity(part, channel_count)` 求解（单一真相源），目标 $\leq 90\%$ RAMB36。35T 上解出：

```
solve_capacity('xc7a35tfgg484-2')
```

```
4096 edges + bank_size 2048 (4096 resident) + UNBOUNDED streaming  
RAMB36: 39/50 = 78%      LUT: 26%   FF: 12%   DSP: 9%
```

三块并行边沿 BRAM (tick/coeff/mask) + 扫描窗口 BRAM 占了 RAMB36 预算的大头；**总线段表是 LUTRAM (distributed)，不占 RAMB36**——因为总线/ramp 引擎每 tick 组合读它们，搬进 BRAM 会破坏组合读。Vivado `report_utilization` / `report_timing_summary` 是最终权威，Python 估算只是预算指引。

9

编译与上传流程

9.1 build / program / run

FPGA 侧只有一个入口 `fpga\textbackslash build\_and\_program.bat` (内部跑 `create_project.tcl` → `program_fpga.tcl`), server 入口是 `fpga\textbackslash run\_server.bat` (jtag-axi 后端):

PowerShell

```
.\fpga\build_and_program.bat --check      # 不连板的 HDL 综合 + 容量自查
.\fpga\build_and_program.bat              # build + program
.\fpga\run_server.bat --check-config      # 打印
↪ project/bit/ltx/xdc/channels/clock/capacity
.\fpga\run_server.bat                    # 启动常驻 hw_axi server (host
↪ 0.0.0.0, port 18861)
```

`create_project.tcl` 生成 `jtag_axi + axi_bram_ctrl` + 5 块 BRAM, 并用 `zlc_force_latency2` 把边沿 BRAM 强制成 `READ_LATENCY_B=2`。产物 `zlc_pulse_streamer_top.\{bit,ltx\}` 落在 `fpga\textbackslash build\textbackslash ps\textbackslash runs\textbackslash impl_1` (工程用短名 `ps` 是为了让 Vivado 深层的 `run / .Xil` 临时路径不超过 Windows MAX_PATH 限制, 同时构建仍然留在仓库内的 `fpga\textbackslash build`, 不挪到别处)。server 加载 `.ltx` 是为了在比特流里定位 `jtag_axi` 核 (即 `hw_axi`), 所以 bit 与 ltx 必须来自**同一次编译**。

9.2 一次上传里发生了什么

1. 主机校验程序: `validate_pulse_streamer_program` 检查边沿/扫描点/位宽/通道不越界, 并校验**全局有效 tick 单调** (扫描不会把合并后的边沿顺序打乱; 会打乱顺序的扫描被拒绝, 而不是悄悄丢点)。
2. `host.image.pack_program` 把整张程序打包成 32-bit BRAM 映像: CTRL 标量 + 三块并行边沿表 + 2-bank 扫描窗口 + 总线段映像。
3. `prepare`: 发 SAFE → AXI 写映像 → arm 两个扫描 bank (写 `BANK_READY/BANK*_CHUNK`) → 发 LOAD (等 `STATUS_LOADED`)。

4. `fire`: 发 FIRE; 对流式/重复扫描, 启动后台回填线程。
5. `wait_done`: 轮询 STATUS (DONE 位) , 并跟随 CURSOR 回填空出来的 bank。
`repeat_forever` 永不置 DONE, 所以必须给 timeout 或主动 stop。

全部在硬件前被 Python 验证

`pack_program`↔`unpack_program` 往返、`engine_model` 的逐周期等价 (1-tick 预取 / 边界无缝 / 流式 ping-pong / 总线 ramp) 、以及 `VivadoAxiStreamerSession` 的 `pack`→`upload`→`LOAD`→`fire` 流程 (Tcl 执行器可注入, 无需 Vivado) 都有契约测试覆盖。仓库没有 Verilog 仿真器, 这套 Python 证据是硬件 bring-up 前的正确性保证。